

บทที่ 6

การพัฒนาโปรแกรมไฟร์สกรีน

6.1 แนวคิด

โปรแกรมไฟร์สกรีนเป็นส่วนของไฟร์วอลล์ที่อยู่บนเกตเวย์ ซึ่งใช้งานระบบปฏิบัติการลินุกซ์ (Gnu/Linux) โดยสำหรับการเข้าใช้บริการของแอคทีฟไดเรกทอรีบนไฟร์สแตชันนั้นจะใช้ OpenLDAP ซึ่งเป็นมาตรฐานเปิดสำหรับใช้งานโปรโตคอล LDAP ซึ่งสามารถใช้งานได้ดีกับทุกแพลตฟอร์ม และในส่วนของการกรองการเชื่อมต่อซึ่งนับเป็นส่วนสำคัญของไฟร์วอลล์แล้วจะใช้ความสามารถของเน็ตฟิลเตอร์/ไอพีเทเบิลส์ (Netfilter/Iptables) ซึ่งทำงานอยู่ในระดับเคอร์เนลและติดตั้งมากับระบบปฏิบัติการลินุกซ์อยู่แล้วตั้งแต่เคอร์เนลเวอร์ชัน 2.4.x ขึ้นมา

6.2 ขอบเขตและความสามารถ

ไฟร์สกรีนทำงานอยู่บนเกตเวย์ ซึ่งการเชื่อมต่อในทุกๆส่วนจะต้องผ่านทางเกตเวย์นี้ นั่นคือไฟร์สกรีนจะสามารถควบคุมการเชื่อมต่อเกือบทั้งหมดในระบบได้ โดยปกติแล้วเกตเวย์ไฟร์วอลล์จะมีลักษณะเป็นแบบ allow-based คือยอมให้แพ็กเก็ตผ่านไปได้อย่างหมด เว้นแต่ส่วนที่ถูกระบุมาตรงกับกฎที่ตั้งไว้ว่าจะไม่ให้ผ่านไป ส่วนการจะตั้งกฎในการที่จะปฏิเสธการเชื่อมต่อหรือแพ็กเก็ตใดบ้างขึ้นขึ้นกับการตั้งค่าของผู้ดูแลระบบผ่านทางไฟร์สแตชัน

6.3 การพัฒนาโปรแกรม

6.3.1 การทำงานร่วมกับไฟร์สแตชัน

ไฟร์สกรีนมีระบบปฏิบัติการเป็นลินุกซ์ ซึ่งในที่นี้ใช้ดิสโทรเดเบียน (debian) โดยการทำงานร่วมกับแอคทีฟไดเรกทอรีของไมโครซอฟท์บนไฟร์สแตชันนั้นจะใช้แพ็คเกจของ OpenLDAP โดยตัวโปรแกรมไฟร์สกรีนจะต้องใช้ไลบรารีของ OpenLDAP ในการเขียนโปรแกรมทำงานร่วมกับแอคทีฟไดเรกทอรี บนระบบปฏิบัติการเดเบียนนั้นใช้แพ็คเกจชื่อ libldap2-dev ซึ่งสนับสนุนโปรโตคอล LDAP เวอร์ชัน 3 และทำงานร่วมกันได้ดีกับแอคทีฟไดเรกทอรี

/	Name	Version	Description
ii	libldap2	2.1.30-12	OpenLDAP libraries
ii	libldap2-dev	2.1.30-12	OpenLDAP development libraries

รูปที่ 6.1 แสดงแพ็คเกจที่จำเป็น

6.3.2 การติดต่อกับแอคทีฟไดเรกทอรี

LDAP *ldap_init(host, port)

char *host;

int port;

ตารางที่ 6.1 พารามิเตอร์ของฟังก์ชัน ldap_init()

host	ชื่อโฮสต์หรือไอพีแอดเดรสของเซิร์ฟเวอร์ (สามารถระบุหลายค่าได้โดยคั่นด้วยช่องว่าง) และสามารถระบุพอร์ตได้ดังนี้ <i>hostname:portnumber</i> ซึ่งจะถือว่าหมายเลขพอร์ตนี้มีความสำคัญกว่า <i>port</i> ซึ่งเป็นพารามิเตอร์ตัวถัดไป
port	หมายเลขพอร์ตที่ใช้ (ค่าปกติคือ 389 หรืออาจใช้ LDAP_PORT แทน)

หลังจากใช้ฟังก์ชัน ldap_init() แล้วการเชื่อมต่อระหว่างไคลเอนต์และเซิร์ฟเวอร์ยังไม่ได้เริ่มขึ้นจริงๆ เพียงคืนค่ากลับมาเป็นเส้นเคิลของการเชื่อมต่อนั้นๆเก็บไว้ในตัวชี้ไปยัง LDAP structure หากไม่มีข้อผิดพลาดเกิดขึ้นเท่านั้น ส่วนการเชื่อมต่อจริงๆจะเริ่มต้นขึ้นในขั้นตอนต่อไป และหากเกิดข้อผิดพลาดขึ้นจะคืนค่ามาเป็น NULL

int ldap_bind_s(LDAP *ld, const char *who, const char *cred, int method);

ตารางที่ 6.2 พารามิเตอร์ของฟังก์ชัน ldap_bind_s()

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
who	dn ของผู้ใช้ที่จะทำงานพิสูจน์ตนเข้าใช้งานแอคทีฟไดเรกทอรี
cred	รหัสผ่านสำหรับผู้ใช้คนนั้นๆ
method	รูปแบบสำหรับระบบการพิสูจน์ตน ปกติใช้เป็น LDAP_AUTH_SIMPLE

หลังจาก ldap_init() แล้วก่อนที่จะกระทำการใดๆกับเซิร์ฟเวอร์ได้จะต้องทำการเริ่มการเชื่อมต่อและพิสูจน์ตนกับเซิร์ฟเวอร์เสียก่อน โดยผ่านทางฟังก์ชัน ldap_bind_s() ซึ่งจะคืนค่า 1 หากเกิดความผิดพลาดขึ้น

_s ที่ต่อท้ายชื่อฟังก์ชันจะหมายถึงเป็นแบบ synchronous คือต้องรอให้ฟังก์ชันทำงานเรียบร้อยได้รับการตอบรับจากเซิร์ฟเวอร์จึงจะสามารถทำคำสั่งต่อไปได้

```
int ldap_unbind(LDAP *ld);
```

```
int ldap_unbind_s(LDAP *ld);
```

ldap_unbind และ ldap_unbind_s ใช้สำหรับจบการเชื่อมต่อกับเซิร์ฟเวอร์ และจะลบค่าใน ld ทั้งหมด โดยทั้ง 2 ฟังก์ชันไม่ได้มีความแตกต่างกันแต่อย่างใด สามารถใช้ได้ไม่ว่าขณะสร้างการเชื่อมต่อได้ใช้ ldap_bind หรือ ldap_bind_s ก็ตาม

ตัวอย่างการเขียนโปรแกรมเชื่อมต่อกับแอคทีฟไดเรกทอรี

```
include <ldap.h>

.....

char *ldap_host = "192.168.1.100";
char *root_dn = "cn=Administrator, cn=Users, dc=hephaestus, dc=com";
char *root_pw = "secret";

.....

LDAP *ld;

.....

/* ldap_init โดยระบุชื่อโฮสต์และพอร์ต */
if ((ld = ldap_init(ldap_host, LDAP_PORT)) == NULL ) {
    perror( "ldap_init failed!!!" );
    exit( EXIT_FAILURE );
}

/* เริ่มการเชื่อมต่อโดยมี credential เป็น root_dn และ root_pw */
if (ldap_bind_s(ld, root_dn, root_pw, LDAP_AUTH_SIMPLE) != LDAP_SUCCESS ) {
```

```

        ldap_perror( ld, "ldap_bind failed!!!" );

        exit( EXIT_FAILURE );

    }

    .....

    /* ยกเลิกการเชื่อมต่อ */
    ldap_unbind_s(ld);

    .....

```

6.3.3 การค้นหาข้อมูลในแอคทีฟไดเรกทอรี

การสืบค้นข้อมูลที่ต้องการบนแอคทีฟไดเรกทอรีเซิร์ฟเวอร์มี 3 ขั้นตอนด้วยกันคือ

1. ได้แต่ละเอนทรีจากผลลัพธ์การค้นหา
2. ได้แอตทริบิวต์ที่ต้องการจากแต่ละเอนทรี
3. ได้ค่าที่เก็บไว้ในแต่ละแอตทริบิวต์

โดยมีฟังก์ชันหลักๆที่ใช้งานดังนี้

int ldap_search_s(ld, base, scope, filter, attrs, attrsonly, res)

DAP *ld;

char *base;

int scope;

char *filter, *attrs[];

int attrsonly;

LDAPMessage **res;

ตารางที่ 6.3 พารามิเตอร์ของฟังก์ชัน ldap_search_s()

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
base	DN ของเอนทรีที่จะเป็นจุดเริ่มต้นของการค้นหา
scope	ขอบเขตของการค้นหา ● LDAP_SCOPE_BASE ค้นหาเอนทรีทั้งหมดบน base

	● LDAP_SCOPE_ONELEVEL ค้นหาเอนทรีทั้งหมดบน base และต่ำลงไป 1 ชั้น ● LDAP_SCOPE_SUBTREE ค้นหาเอนทรีทั้งหมดบน base และได้เรกทอรีภายใต้ทั้งหมด
filter	ฟิลเตอร์ที่จะใช้กรองผลลัพธ์จากการค้นหา อยู่ในรูปแบบ attributetype=attributevalue
attrs	อาร์เรย์ของชนิดแอตทริบิวต์ที่จะส่งคืนมาสำหรับเอนทรีที่เข้าข่าย
attrsonly	ระบุว่าค่าในแอตทริบิวต์จะถูกส่งกลับมามีค่าด้วยหรือไม่ ● 0 หมายความว่าทั้งแอตทริบิวต์และค่าของแอตทริบิวต์นั้นถูกส่งกลับ ● 1 หมายความว่าเพียงแอตทริบิวต์เท่านั้นที่ถูกส่งกลับ
res	ผลลัพธ์จากการค้นหา

LDAPMessage *ldap_first_entry(LDAP *ld, LDAPMessage *result);

ตารางที่ 6.4 พารามิเตอร์ของฟังก์ชัน ldap_first_entry()

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
result	ชุดของผลลัพธ์ในรูปแบบของ LDAPMessage structure

char *ldap_first_attribute(LDAP *ld, LDAPMessage entry, BerElement **berptr);

ตารางที่ 6.5 พารามิเตอร์ของฟังก์ชัน ldap_first_attribute()

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
entry	ตัวชี้ไปยังเอนทรีที่คืนค่ามาจาก ldap_first_entry() โดยอยู่ในรูปแบบของ LDAPMessage structure
berptr	ตัวชี้ไปยัง BerElement ใช้สำหรับการจดจำตำแหน่งของแอตทริบิวต์ต่างๆในเอนทรีที่ได้มา

char **ldap_get_values(ld, entry, attr)

LDAP *ld;

LDAPMessage *entry;

char *attr

ตารางที่ 6.6 พารามิเตอร์ของฟังก์ชัน ldap_get_values()

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
entry	ผลลัพธ์จาก ldap_search_s()
attr	ค่า (value) ในแอตทริบิวต์จาก ldap_first_attribute()

ตัวอย่างการค้นหาข้อมูลที่ต้องการบนแอตทริบิวต์ไฟร์วอลล์

```

include <ldap.h>

include <lber.h>

.....

LDAP *ld;

char *base = "cn=computers, dc=hephaestus, dc=com";

char *filter = "firescreen";

char* attr = "firewallRule";

char** vals;

BerElement* ber;

LDAPMessage* msg;

LDAPMessage* entry;

.....

/* ldap_search_s จะคืนเอนทรีที่เข้าข่ายสำหรับขอบเขตการค้นหาและฟิลเตอร์ที่ระบุเป็น
ตัวชี้ในตัวแปร msg */

if (ldap_search_s(ld, base, LDAP_SCOPE_SUBTREE, filter, &attr, 0, &msg) !=
LDAP_SUCCESS) {

    ldap_perror( ld, "ldap_search_s failed!!!" );

    exit(EXIT_FAILURE);

}

```

```

/* ลูป for สำหรับวนจนครบทุกๆเอนทรีในส่งกลับมาจาก ldap_search_s */
for (entry = ldap_first_entry(ld, msg); entry != NULL; entry = ldap_next_entry(ld,
entry)) {

    /* ลูป for สำหรับวนจนครบทุกแอตทริบิวต์ในแต่ละเอนทรี */
    for( attr = ldap_first_attribute(ld, entry, &ber);
        attr != NULL; attr = ldap_next_attribute(ld, entry, ber)) {

        /* ลูป for สำหรับวนจนครบทุกๆค่าในแต่ละแอตทริบิวต์ */

        if ((vals = ldap_get_values(ld, entry, attr)) != NULL) {
            /* ค่าที่ต้องการจะอยู่ใน vals[i] */

        }

    }

}

}

.....

```

6.3.4 การเปลี่ยนแปลงแก้ไขข้อมูลบนแอคทีฟไดเรกทอรี

การเปลี่ยนแปลงแก้ไขข้อมูลที่ต้องการในแอคทีฟไดเรกทอรีเซิร์ฟเวอร์นั้นสามารถทำได้ผ่านฟังก์ชัน `ldap_modify_s()`

```
int ldap_modify_s(ld, dn, mods)
```

```
LDAP *ld;
```

```
char *dn;
```

```
LDAPMod *mods[];
```

ตารางที่ 6.7 พารามิเตอร์ของฟังก์ชัน `ldap_modify_s()`

ld	ตัวชี้ไปยัง LDAP structure ที่ทำงานด้วย
dn	DN ของเอนทรีที่จะทำการแก้ไขเปลี่ยนแปลง
mods	ตัวชี้ไปยังอาร์เรย์ของตัวชี้ไปยัง LDAPMod structure สำหรับระบุว่า จะทำการเปลี่ยนแปลงอย่างไร

ตัวอย่างการเปลี่ยนแปลงแก้ไขข้อมูลบนแอคทีฟไดเรกทอรี

```

include <ldap.h>

.....

LDAP *ld;
LDAPMod *list_of_attrs[4];
LDAPMod attribute1, attribute2, attribute3;

char *dn = "cn=firescreen, cn=computers, dc=hephaestus, dc=com";

/* คำที่จะทำการแก้ไขเปลี่ยนแปลง */
char *homePhone_values[] = { "555-1212", NULL };
char *telephoneNumber_values[] = { "869-5309", NULL };

.....

/* ระบุรูปแบบการเปลี่ยนแปลงแก้ไขในแต่ละแอตทริบิวต์ */
attribute1.mod_type = "homePhone";
attribute1.mod_op = LDAP_MOD_ADD;
attribute1.mod_values = homePhone_values;
attribute2.mod_type = "telephoneNumber";
attribute2.mod_op = LDAP_MOD_REPLACE;
attribute2.mod_values = telephoneNumber_values;
attribute3.mod_type = "facsimileTelephoneNumber";
attribute3.mod_op = LDAP_MOD_DELETE;
attribute3.mod_values = NULL;

/* เก็บแต่ละคำที่จะเปลี่ยนแปลงแก้ไขในอาร์เรย์ */
list_of_attrs[0] = &attribute1;
list_of_attrs[1] = &attribute2;

```



```

list_of_attrs[2] = &attribute3;

list_of_attrs[3] = NULL;

.....

/* เปลี่ยนแปลงแก้ไขในแต่ละแอตทริบิวต์ */

if ( ldap_modify_s( ld, dn, list_of_attrs ) != LDAP_SUCCESS ) {

    ldap_perror( ld, "ldap_modify_s" );

    return( 1 );

}

.....

```

mod_type, mod_op และ mod_values ถูกนิยามไว้ใน LDAPMod structure โดย mod_type บ่งบอกถึงแอตทริบิวต์ที่ต้องการทำงานด้วยและ mod_op คือรูปแบบการเปลี่ยนแปลงแก้ไข ประกอบไปด้วย LDAP_MOD_ADD, LDAP_MOD_REPLACE และ LDAP_MOD_DELETE ส่วน mod_values ก็คือค่าที่จะทำการแก้ไขเปลี่ยนแปลง หาก mod_op เป็น LDAP_MOD_DELETE นั้นก็จะต้องให้ค่า mod_values เป็น NULL

6.3.5 การนำกฎจากไฟร์สแตนด์มาใช้งานไฟร์สกรีน

6.3.5.1 การดึงกฎจากไฟร์สแตนด์

สำหรับกฎของไฟร์วอลล์ที่เก็บไว้ในแอตทริบิวต์ไฟร์สแตนด์นั้นถูกเก็บไว้ได้ dn คือ "cn=firescreen, cn=computers, dc=hephaestus, dc=com" ภายในแอตทริบิวต์ชื่อ firewallRule ดังนั้นจึงสามารถเข้าถึงได้โดยใช้ฟังก์ชัน ldap_search_s() กำหนดขอบเขตของการค้นหาเป็น "cn=firescreen, cn=computers, dc=hephaestus, dc=com" จากนั้นจึงดึงค่าออกจากแอตทริบิวต์ firewallRule จนครบทุกค่าและเก็บค่าไว้ใช้งานในอาร์เรย์

จากรูปแบบการทำงานของไฟร์สกรีนที่กล่าวถึงในหัวข้อก่อนหน้านี้ ไฟร์สกรีนจะคอยตรวจสอบที่ไฟร์สแตนด์ทุกระยะเวลาหนึ่ง เพื่อตรวจสอบว่ามีกฎถูกแก้ไขหรือไม่ ซึ่งหลังจากกฎถูกแก้ไขบนไฟร์สแตนด์จะทำการอัปเดตค่าในแอตทริบิวต์ชื่อ update เป็น "YES" ดังนั้นก่อนหน้านี้ที่จะรับกฎมาจากไฟร์สแตนด์จึงต้องทำการตรวจสอบแอตทริบิวต์ update ก่อน หากมีค่าเป็น "NO" ก็จบการทำงานและรอเวลาวนกลับไปตรวจสอบอีกครั้ง และหากไฟร์สกรีนพบว่าค่าในแอตทริ

บิวต์ update เป็น "YES" ก็จะทำการดึงกฎมาจากแอตทริบิวต์ firewallRule จากนั้นจึงทำการแก้ไขค่าในแอตทริบิวต์กลับเป็น "NO" อีกครั้งผ่านทางฟังก์ชัน ldap_modify_s()

6.3.5.2 การนำมาใช้งานกับไอพีเทเบิลส์

กฎที่ดึงมาจากทางไฟร์สแตชันนั้นจะอยู่ในรูปแบบดังต่อไปนี้

SRC:MASK:DST:MASK:PROTOCOL:SPORT:DPORT:ICMP-TYPE:ZONE:TARGET

ตารางที่ 6.8 แสดงความหมายของแต่ละค่า

SRC	ไอพีแอดเดรสต้นทาง
MASK	เน็ตมาสก์ของไอพีแอดเดรสต้นทาง
DST	ไอพีแอดเดรสปลายทาง
MASK	เน็ตมาสก์ของไอพีแอดเดรสปลายทาง
PROTOCOL	โปรโตคอล (TCP, UDP, ICMP หรือ any)
SPORT	พอร์ตต้นทาง (เมื่อระบุโปรโตคอลเป็น TCP หรือ UDP เท่านั้น)
DPORT	พอร์ตปลายทาง (เมื่อระบุโปรโตคอลเป็น TCP หรือ UDP เท่านั้น)
ICMP-TYPE	หมายเลข ICMP-TYPE (เมื่อระบุโปรโตคอลเป็น ICMP เท่านั้น)
ZONE	โซนที่กฎนี้จะมีผลในการใช้งาน
TARGET	ปลายทางของแพ็กเก็ตที่แมชท์กับกฎนี้

โดยก่อนที่จะสามารถนำมาใช้งานกับไฟร์สกรีนได้จะต้องแปลงกฎที่รับมาจากไฟร์สแตชันที่มีรูปแบบดังกล่าวให้อยู่ในรูปแบบของไอพีเทเบิลส์เสียก่อน โดยผ่านฟังก์ชันแปลง (parser) ซึ่งจะแยกแต่ละส่วนออกจากกันโดยมีเครื่องหมายโคลอน (:) เป็นตัวแบ่ง โดยนำเอาแต่ละส่วนมาต่อเข้าด้วยกันตามรูปแบบคำสั่งของไอพีเทเบิลส์โดยขึ้นต้นด้วย iptables -A (กฎจะถูกเพิ่มต่อท้ายเรื่อยๆ ตามลำดับ) ส่วนกฎจะถูกเพิ่มเข้าเชน (CHAIN) โดยของไอพีเทเบิลส์จะพิจารณาจาก ZONE และ TARGET ที่ระบุเป็น DROP ทั้งหมดจะถูกระบุ TARGET ในคำสั่งไอพีเทเบิลส์เป็น -j LOG-CHAIN ซึ่งจะส่งแพ็กเก็ตที่ตรงกับกฎเข้าไปยังเชนชื่อ LOG-CHAIN เพื่อที่หากแพ็กเก็ตที่ถูกละทิ้ง (DROP) จะมีการเก็บข้อมูลลงล็อกไฟล์

```
Chain LOG-CHAIN (4 references)
num target prot opt source destination limit: avg 1/sec
1 LOG all -- anywhere anywhere
burst 5 LOG level info prefix `FIRESCREEN '
2 DROP _ all -- anywhere anywhere
```

รูปที่ 6.2 กฎในเชน LOG-CHAIN

กฎแรกจะทำการเก็บข้อมูลทุกแพ็คเก็ตที่ผ่านเข้ามาในเซสนี้ลงล็อกไฟล์ในอัตรา 1 แพ็คเก็ตต่อวินาที (limit: avg 1/sec burst 5) โดยทุกๆค่าที่ถูกเก็บลงล็อกไฟล์จะมีคำว่า "FIREScreen " (prefix 'FIREScreen ') ขึ้นต้น เพื่อประโยชน์ในการคัดแยกล็อกในภายหลัง เนื่องจากกระบวนการเก็บเหตุการณ์ลงล็อกไฟล้นั้นถูกจัดการโดย syslogd ซึ่งไม่ได้ถูกจัดการโดยตัวไอฟีเทเบิลส์เองทำให้ในล็อกไฟล์จะเก็บเหตุการณ์ที่มีระดับของการล็อกเหมือนกันเอาไว้ โดยในเซส LOG-CHAIN นี้ระดับของการล็อกเป็น info (LOG level info)

กฎที่สองจะทำการละทิ้งหมดทุกๆแพ็คเก็ต เนื่องด้วยแพ็คเก็ตที่ถูกส่งต่อมายังเซสนี้เพื่อเก็บข้อมูลลงล็อกไฟล์คือแพ็คเก็ตที่ถูกกำหนด TARGET จากไฟร์สเตชันไว้เป็น DROP นั่นเอง

ตัวอย่างการแปลงกฎจากไฟร์สเตชันมาเป็นคำสั่งของไอฟีเทเบิลส์

กฎจากไฟร์สเตชัน 192.168.2.10:255.255.255.255:192.168.1.2:255.255.255.255:TCP:ANY:22:-:DMZ-in:DROP คำสั่งไอฟีเทเบิลส์ iptables -A FORWARD -s 192.168.2.10 -d 192.168.1.2 -p tcp -dport 22 -j LOG-CHAIN

กฎจากไฟร์สเตชัน 192.168.2.7:255.255.255.255::-:ANY::-:LOCAL-in:ACCEPT คำสั่งไอฟีเทเบิลส์ iptables -A INPUT -s 192.168.2.7 -j ACCEPT

สุดท้ายเมื่อได้กฎจากไฟร์สเตชันอยู่ในรูปแบบคำสั่งของไอฟีเทเบิลส์ทั้งหมดแล้วก็จะทำการเขียนทั้งหมดลงบนไฟล์เชลล์สคริปต์ เพื่อทำการรันเพื่อใช้งานตามคำสั่งไอฟีเทเบิลส์ดังกล่าว รวมทั้งเก็บไฟล์นี้ไว้ใช้ครั้งต่อไปหากโปรแกรมถูกปิดหรือเครื่องไฟร์สกรีนเริ่มทำงานใหม่เพื่อที่จะสามารถคงกฎเดิมไว้ได้

6.3.6 การส่งล็อกกลับไปยังไฟร์สเตชัน

ไฟร์สเตชันจะมีการเก็บล็อกไฟล์สำหรับแพ็คเก็ตที่ถูกทิ้งไปตามกฎที่ได้กำหนดไว้เอาไว้เพื่อใช้ประกอบการพิจารณาป้องกันการบุกรุกที่เกิดขึ้นหรือความผิดปกติที่อาจเกิดขึ้น ซึ่งล็อกไฟล์จะถูกเก็บลงไฟล์บนเครื่องไฟร์สกรีนเองก่อน จากนั้นจึงค่อยส่งต่อไปเก็บยังศูนย์กลางที่ไฟร์สเตชันต่อไป

ล็อกไฟล์ในส่วนของไอพีเทเบิลส์นั้นไม่ได้ถูกเก็บไว้โดยตัวไอพีเทเบิลส์เองโดยตรง แต่ผ่านโปรแกรมเก็บล็อกมาตรฐานบนระบบปฏิบัติการยูนิกซ์อย่าง syslogd ล็อกไฟล์ในประเภทเดียวกันก็จะถูกเก็บไว้ในไฟล์เดียวกัน โดยปกติล็อกในส่วนของไอพีเทเบิลส์จะถูก syslogd เก็บไว้ที่ /var/log/kern.log แต่สำหรับบนตัวไฟร์สกรีนเองนั้นได้แยกล็อกไฟล์ในส่วนนี้เก็บแยกออกไปต่างหากที่ /var/log/firescreen.log อีกส่วนด้วย ซึ่งแยกออกตามระดับการล็อก โดยได้กำหนดระดับของล็อกไว้ที่ info อย่างไรก็ตามล็อกในส่วนของเคอร์เนลที่มีระดับเป็น info เช่นกันก็จะยังถูกเก็บลงยังไฟล์ firescreen.log นี้ด้วยเช่นกัน ดังนั้นแล้วขณะทำการล็อกจะกำหนดคำนำหน้า (prefix) ในส่วนของล็อกจากไอพีเทเบิลส์เป็น "FIRESCREEN " เพื่อความง่ายในการแยกล็อกของไอพีเทเบิลส์ออกจากล็อกจากส่วนอื่นๆ

```
Jan 10 16:00:45 firescreen kernel: NET: Registered protocol family 10
Jan 10 16:00:45 firescreen kernel: Disabled Privacy Extensions on device c02cc960(lo)
Jan 10 16:00:45 firescreen kernel: IPv6 over IPv4 tunneling driver
Jan 10 16:01:53 firescreen kernel: FIRESCREEN IN=eth0 OUT=eth1 SRC=161.246.5.11
DST=192.168.1.100 LEN=40 TOS=0x00 PREC=0x00 TTL=1 ID=38473 PROTO=UDP SPT=38469
DPT=33438 LEN=20
Jan 10 16:01:58 firescreen kernel: FIRESCREEN IN=eth0 OUT=eth1 SRC=161.246.5.11
DST=192.168.1.100 LEN=40 TOS=0x00 PREC=0x00 TTL=1 ID=38474 PROTO=UDP SPT=38469
DPT=33439 LEN=20
```

รูปที่ 6.3 แสดงล็อกที่ถูกเก็บไว้โดย syslogd

รูปที่ 6.3 แสดงให้เห็นข้อมูลในล็อกไฟล์ /var/log/firescreen.log ซึ่งกำหนดไว้ในไฟล์การตั้งค่าของ syslogd เป็น kern.info -/var/log/firescreen.log จะพบว่าไม่เฉพาะเพียงแต่ล็อกในส่วนของไอพีเทเบิลส์เท่านั้นที่ถูกเก็บไว้ในไฟล์นี้แต่จะมีล็อกจากเหตุการณ์อื่นๆที่อยู่ในระดับ info รวมอยู่ด้วยเช่นกัน ล็อกของไอพีเทเบิลส์เองสามารถสังเกตได้จากคำนำหน้า "FIRESCREEN " หลังส่วนต้นที่แสดงเวลาและชื่อโฮสต์

โปรแกรมจะคอยอ่านล็อกไฟล์ดังกล่าวเป็นระยะและส่งกลับไปเก็บยังแอคทิฟไดเรกทอรีบนไฟร์สแตชั่น จากนั้นจะทำการลบข้อมูลทั้งไฟล์ทิ้งเพื่อตัดปัญหาที่จะส่งล็อกเดิมไปอีกครั้งหนึ่ง โดยภายใต้ dn "cn=firescreen, cn=computers, dc=hephaestus, dc=com" ในแอคทิฟไดเรกทอรีบนไฟร์สแตชั่นนั้นจะมีแอตทริบิวต์ชื่อ firewallLog อยู่สำหรับเก็บล็อกที่ส่งมาจากไฟร์สกรีน อนึ่ง การเพิ่มล็อกเข้าไปยังแอตทริบิวต์ดังกล่าวสามารถทำได้ผ่านฟังก์ชัน ldap_modify_s() ดังกล่าวถึงในหัวข้อก่อนหน้า

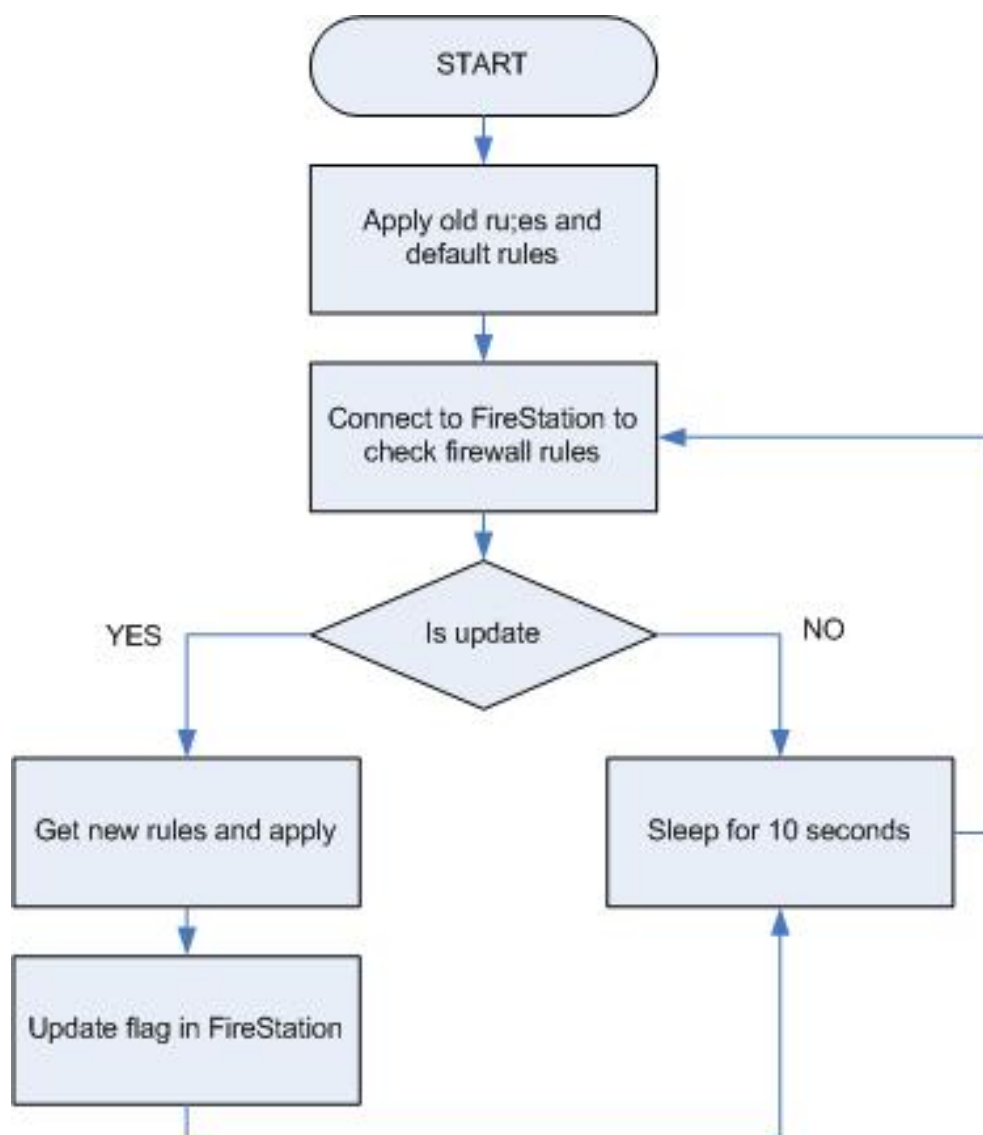
6.4 การทำงานของโปรแกรม

6.4.1 ลักษณะการทำงานของไฟร์สกรีน

การทำงานของไฟร์สกรีนจะเป็นในลักษณะของโปรแกรมเดมอน (daemon) คือทำงานอยู่เบื้องหลังตลอดเวลาตั้งแต่เครื่องไฟร์สกรีน (เกตเวย์) เริ่มทำงานขึ้นมา โดยการทำงานนั้นจะแบ่งออกเป็น 2 ส่วนหลักคือ ส่วนที่ทำหน้าที่รับกฎจากไฟร์สเตชันมาใช้ และส่วนของการส่งล็อกกลับไปยังไฟร์สเตชัน

1. ส่วนที่ทำหน้าที่รับกฎจากไฟร์สเตชัน

โปรแกรมในส่วนนี้จะทำการนำเอาที่ถูกกำหนดไว้ที่ไฟร์สเตชันซึ่งถูกเก็บไว้ในแอคทีฟไดเรกทอรีมาใช้งานกับไอพีเทเบิลส์ สามารถแสดงลักษณะการทำงานเป็นแผนภาพดังนี้

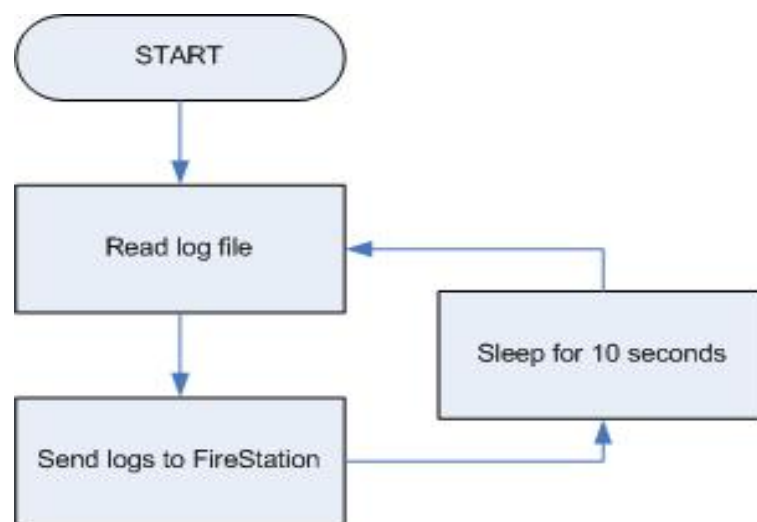


รูปที่ 6.4 แสดงการรับกฎของไฟร์สกรีน

เมื่อเริ่มทำงานขึ้นมาครั้งแรกไฟร์สกรีนจะทำการตั้งกฎพื้นฐานที่ถูกกำหนดไว้แล้วให้กับ ไอพีเทเบิลส์ก่อน และจากนั้นจึงตรวจสอบดูกฎที่ถูกเก็บไว้เดิมและนำมาใช้งานกับไอพีเทเบิลส์เป็น ที่เรียบร้อยแล้วเสียก่อน จากนั้นจึงตรวจสอบดูที่ไฟร์สเตชั่นว่าจากที่รับกฎมาครั้งล่าสุดได้มีการ เปลี่ยนแปลงแก้ไขหรือไม่ หากมีก็จะนำกฎที่กำหนดใหม่มาใช้ หากไม่มีก็จะหยุดระยะเวลาหนึ่ง แล้วจึงคอยตรวจสอบที่ไฟร์สเตชั่นอีกเรื่อยๆ (polling)

2. ส่วนที่ทำหน้าที่ส่งล็อกไปยังไฟร์สเตชั่น

โปรแกรมในส่วนนี้จะทำการอ่านล็อกไฟล์จากเครื่องเกตเวย์และส่งไปเก็บยังแอคทิไฟด์ เรกทอรีบนไฟร์สเตชั่น สามารถแสดงลักษณะการทำงานเป็นแผนภาพดังนี้



รูปที่ 6.5 แสดงการส่งล็อกของไฟร์สกรีน

เมื่อเริ่มการทำงานขึ้นมาโปรแกรมจะคอยตรวจสอบล็อกไฟล์เป็นระยะ และส่งล็อกที่ เกี่ยวข้องในส่วนของไอพีเทเบิลส์กลับไปยังไฟร์สเตชั่น

6.4.2 กฎพื้นฐานบนไฟร์สกรีน

ไฟร์สกรีนเองจะต้องมีการทำงานร่วมกันกับไฟร์สเตชันในการรับกฎและส่งสื่อกลับไป ดังนั้นกฎที่ใช้งานบนไฟร์สกรีนแม้รับมาจากตัวไฟร์สเตชัน แต่บนตัวไฟร์สกรีนเองโปรแกรมจะต้องมีการตั้งกฎพื้นฐานไว้ส่วนหนึ่งก่อนหน้ากฎอื่นๆเพื่อเป็นการป้องกันปัญหาที่อาจเกิดขึ้นหากมีการตั้งกฎที่เป็นปัญหากับการทำงานขึ้นในภายหลัง เช่นอาจทำให้ไฟร์สกรีนไม่สามารถทำการติดต่อไปยังไฟร์สเตชันได้ เป็นต้น นอกเหนือไปจากนั้นยังเป็นเพื่อรับประกันความปลอดภัยพื้นฐานให้กับตัวไฟร์สกรีนและตัวระบบเองด้วยเช่นกัน

รองรับการสามารถติดต่อเข้าไปยังไฟร์สเตชันจากเครื่องไคลเอนต์ภายใน

```
iptables -A FORWARD -s $LOCAL -d $FIRESTATION -p tcp --dport 686 -i $LOCAL_INTERFACE -o $DMZ_INTERFACE -j ACCEPT
iptables -A FORWARD -s $FIRESTATION -d $LOCAL -p tcp -m state --state ESTABLISHED,RELATED -i $DMZ_INTERFACE -o $LOCAL_INTERFACE -j ACCEPT
```

รองรับการติดต่อกลับจากไฟร์สเตชัน

```
iptables -A FORWARD -s $DMZ -m state --state ESTABLISHED,RELATED -i $DMZ_INTERFACE -j ACCEPT
```

ป้องกันการโจมตีจาก syn flood โดยจำกัดปริมาณการเชื่อมต่อที่มี TCP flag เป็น syn ไว้ที่ 5 ครั้งต่อ 1 วินาที

```
iptables -A FORWARD -p tcp --syn -m limit --limit 5/second -j ACCEPT
```

ละทิ้ง (DROP) แพ็คเก็ตที่พบว่า invalid ทั้งหมด

```
iptables -A FORWARD -s 224.0.0.0/4 -j DROP
iptables -A FORWARD -s 240.0.0.0/5 -j DROP
iptables -A FORWARD -s 127.0.0.0/8 -j DROP
iptables -A FORWARD -s 169.254.0.0/16 -j DROP
iptables -A FORWARD -d 224.0.0.0/4 -j DROP
iptables -A FORWARD -d 240.0.0.0/5 -j DROP
iptables -A FORWARD -d 127.0.0.0/8 -j DROP
iptables -A FORWARD -d 169.254.0.0/16 -j DROP
iptables -A FORWARD -d 255.255.255.255 -j DROP
iptables -A FORWARD -m state --state INVALID -j DROP
```

รองรับการเชื่อมต่อเข้ามายัง localhost ทั้งหมด

```
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
```

กฎที่กำหนดให้กับเชน FORWARD ทั้งหมดจะต้องใส่ให้กับเชน INPUT และ OUTPUT เพื่อรองรับการติดต่อระหว่างไฟร์สกรีนเองกับไฟร์สเตชันและป้องกันความปลอดภัยของตัวไฟร์สกรีนเองด้วยเช่นกัน